

App Development Project Report

Modern Application Development (MAD) 2

1. Student Details

Name: Harsh Dhiman

Roll Number: 24f2000016

Email: 24f2000016@ds.study.iitm.ac.in

About Me: I am a student at IIT Madras BS Degree program with a deep interest in web application development and data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

2. Project Details

Project Title: Hospital Management App (MAD2)

Problem Statement:

To design and build a modern web-based application that allows patients, doctors and management to manage appointments, patients' databases and doctors' databases more efficiently than traditional (manual) ways.

Approach:

The app was built using Flask as the backend framework and VueJS as frontend framework. It allows hospital management to manage patients and doctors data more efficiently and allows patients to book appointments in a better way.

3. AI/LLM Declaration

I used GitHub Copilot available in VS code to assist in styling and structuring vue components. The extent of AI/LLM usage is around **15-20%**, limited to **code suggestions and CSS styling**.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Vue.js	Framework for all the core frontend logic
FlaskSecurity	Framework for generating and validating login tokens, RBAC and storing passwords securely
SQLite	Lightweight local database for storing user data
SQLAlchemy	Object Relational Mapper for SQLite database
Flask	Core backend web framework

5. Database Schema / ER Diagram

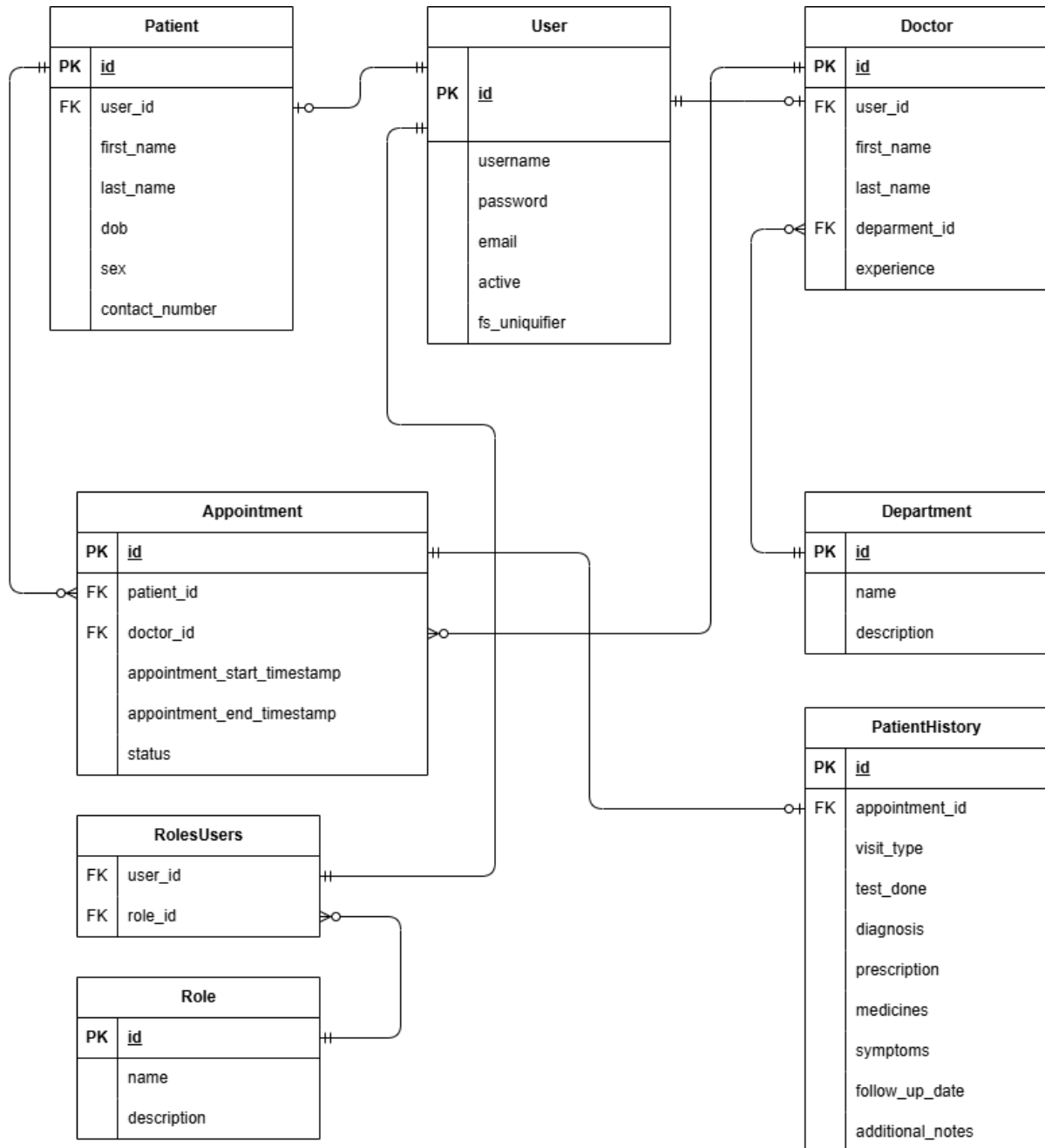
Tables:

1. **User** — stores users profile details (id, username, password, email, active, fs_uniquifier)
2. **Department** — stores departments details (id, name, description)
3. **Patient** — stores patients details (id, user_id, first_name, last_name, dob, sex, contact_number)
4. **Doctor** — stores doctors details (id, user_id, first_name, last_name, department_id, experience)
5. **Appointment** — stores appointments details between doctors and patients (id, patient_id, doctor_id, appointment_start_timestamp, appointment_end_timestamp, status)
6. **RolesUsers** — stores relationship between users and roles (user_id, role_id)
7. **PatientHistory** — stores data of completed appointments (id, appointment_id, visit_type, test_done, diagnosis, prescription, medicines, symptoms, follow_up_date, additional_notes)
8. **Role** — stores roles details (id, name, description)

Relationships:

- One-to-One → User → Patient
- One-to-One → User → Doctor
- One-to-One → PatientHistory → Appointment
- One-to-Many → Department → Doctor
- One-to-Many → Patient → Appointment
- One-to-Many → Role → RoleUsers
- One-to-Many → Doctor → Appointment
- One-to-One → Appointment → PatientHistory

- One-to-One → User → RoleUsers



6. API Resource Endpoints

Endpoint	Method	Description
----------	--------	-------------

<code>/api/login</code>	POST	Authenticate Patient and return authentication token
<code>/api/register</code>	POST	Patient can register themselves
<code>/api/current-user-details</code>	GET	Takes authentication token and returns details (email, username, roles) of current logged in user
<code>/api/admin-dashboard</code>	GET	Return patient, doctor, department and appointment data
<code>/api/doctor</code>	POST	Add a new doctor
<code>/api/doctor</code>	GET	Send all doctors data
<code>/api/doctor/<int: doctor_id></code>	PUT	Update details of a registered doctor
<code>/api/doctor/<int: doctor_id>/blacklist</code>	POST	Blacklist a registered doctor
<code>/api/doctor/<int: doctor_id>/whitelist</code>	POST	Undo blacklisting a doctor
<code>/api/patient</code>	POST	Admin can directly register a patient
<code>/api/patient</code>	GET	Send data of all patients
<code>/api/patients/<int: patient_id></code>	PUT	Admin can update/modify patients profile details
<code>/api/patient/<int: patient_id>/blacklist</code>	POST	Blacklist a patient
<code>/api/patient/<int: patient_id>/whitelist</code>	POST	Undo blacklist a patient
<code>/api/profile</code>	PUT	Patient can update their profile details
<code>api/department</code>	GET	Get details of all departments

<code>api/department</code>	POST	Admin can create new department
<code>api/appointment</code>	GET	Get data of all appointments
<code>api/appointment</code>	POST	Patients can book appointments
<code>api/appointment/<int: appointment_id></code>	GET	Get details of a particular appointment
<code>/api/patient/my_appointments</code>	GET	Particular patient can get details of about all their appointments
<code>/api/appointments/<int: appointment_id></code>	PUT	Admin can modify appointment
<code>/api/appointment/available-slots/<int: doctor_id>/<date_start></code>	GET	Get available appointment slots for a doctor on specific date (10-min intervals)
<code>/api/appointment/<int: appointment_id></code>	PUT	Patients can update their own appointment Doctors can update appointments they are assigned to.
<code>/api/appointments/<int: appointment_id>/complete</code>	PUT	Complete an appointment and save data to patient history
<code>/api/doctor/unavailable-slots</code>	POST	Doctor blocks a time slot for unavailability
<code>/api/doctor/unavailable-slots/<int: doctor_id></code>	GET	Get all unavailable slot for a doctor
<code>/api/doctor/unavailable/<int: slot_id></code>	DELETE	Doctor removes an unavailable slot
<code>/api/patient/<int: patient_id>/medical-history</code>	GET	Get medical history of a patient

/api/my/medical-history GET

Patients can view their own medical history

7. Video Presentation

Drive Link:

https://drive.google.com/file/d/1ZLI8JYSEOc-1LI2ZdIBCDXr-o-z_vSJK/view?usp=drive_link

 HarshDhiman-AppDev2-Project.mp4
